

# LocalSearchAgent 规划策略、工具链路与异常处理设计

## 背景与目标

LocalSearchAgent 面向本地生活行程规划：用户用自然语言提出需求，系统需要在有限供给、用户偏好和可执行 API 之间形成稳定闭环。

目标：

- 把口语需求转成可被模型消费的结构化约束，降低下游重新理解自然语言的风险。
- 将规划拆成可观测的 A/B/C 阶段，保证每个节点只承担清晰职责，并通过 PlanState 传递契约。
- 引入 Value Probe，将亲子照顾、健康饮食、便利省心、预算敏感等隐含价值信号量化，并作为 Memory、排序权重和解释链路的输入。

## 组件目的

| 阶段      | 组件                                           | 组件目的                                                                                                  |
|---------|----------------------------------------------|-------------------------------------------------------------------------------------------------------|
| A 用户理解层 | `intent_parser`                              | 解决“用户到底要什么”。把自然语言解析为`intent`、`constraints`、`scene_type`、时间窗、人数、地点、预算、偏好和`avoid`；LLM 只做增强，失败时保留规则解析结果。 |
|         | `memory_manager`                             | 解决“用户有哪些稳定偏好”。读取长/短期 Memory，合并价值观、风险和历史反馈；当前输入优先于历史 Memory。                                           |
|         | `scenario_planner`                           | 解决“需求属于什么场景”。把 intent 与 memory 转成场景模板、活动标签、节奏、POI mix 和路线倾向，为 B 提供搜索边界。                               |
| B 规划优化层 | `b_poi_rag`                                  | 解决“大量 POI 如何快速召回”。按城市、类目、景角色和约束检索候选，缩小候选池，不直接决定最终方案。                                                  |
|         | `candidate_generator`                        | 解决“哪些商家组合可进入规划”。生成活动、餐饮等候选，并做城市、距离、时间、忌口、儿童友好等初筛。                                                     |
|         | `constraint_filter`                          | 解决“哪些方案违反硬约束”。过滤时间、距离、预算、排队、库存、同行人数适配或`avoid`冲突的候选。                                                   |
|         | `b_itinerary_blueprint`                      | 解决“行程应该是什么形态”。识别半天、全天、过夜、两天等需求，拆成活动、餐饮、住宿、购物等节点骨架。                                                    |
|         | `plan_optimizer`                             | 解决“哪个方案最值得推荐”。综合路线、预算、偏好、同行人数、履约可用和风险惩罚评分，输出主方案、备选和解释。                                                |
|         | `b_replan_loop` /<br>`b_repair_planner`      | 解决“失败后如何局部修复”。当库存、订座或时间窗失败时，保留可用节点，只替换失败节点并触发重排。                                                      |
| C 执行闭环层 | `b_ai_plan_critic`                           | 解决“规则难以穷举的体验风险”。LongCat 检查方案是否贴合真实意图和同行人数需求，只能提出有边界的 replan 建议。                                       |
|         | `tool_router`                                | 解决“规划结果该调用哪些工具”。把`selected_plan`转成查库存、订票、订座、买券、支付和分享等动作。                                              |
|         | `mock_api_adapter` /<br>`execution_mock_api` | 解决“如何模拟真实履约”。提供库存、票务、订座、优惠券和订单 mock API，返回执行状态和失败原因。                                                  |
|         | `share_generator`                            | 解决“如何生成可转发消息”。基于已确认的时间、地点、路线、预算和执行结果生成同行人数分享文案。                                                       |
| 横向能力    | Frontend                                     | 解决“用户如何理解并修改规划”。展示需求理解、Memory、地图、方案、解释和执行状态，并支持用户编辑后重新规划。                                             |
|         | Eval & Observability                         | 解决“如何知道系统变好了”。记录耗时、候选数量、过滤原因、RAG 命中、replan、得分和失败类型，用 eval 与人工测试验证效果。                                  |

## Planning 策略

规划采用先收窄语义，再生成候选，再执行验证的分层策略：

- A-stage 解析 intent、合并 memory、生成 scenario handoff。该阶段输出结构化事实和偏好，不编造商家、价格、库存或预约结果。
- B-stage 使用 b\_poi\_rag、candidate\_generator、constraint\_filter、plan\_optimizer 和 b\_replan\_loop。硬约束优先过滤，软偏好进入排序；若候选证据不足，返回 needs\_rag\_candidate\_evidence 或发起 repair/replan。
- C-stage 只执行 B-stage 标记为 ready 的计划。tool\_router 将 timeline 转为 reserve\_restaurant、order\_activity\_ticket 等 action；执行层校验库存、时间、人数、路线、支付和分享。

核心原则：

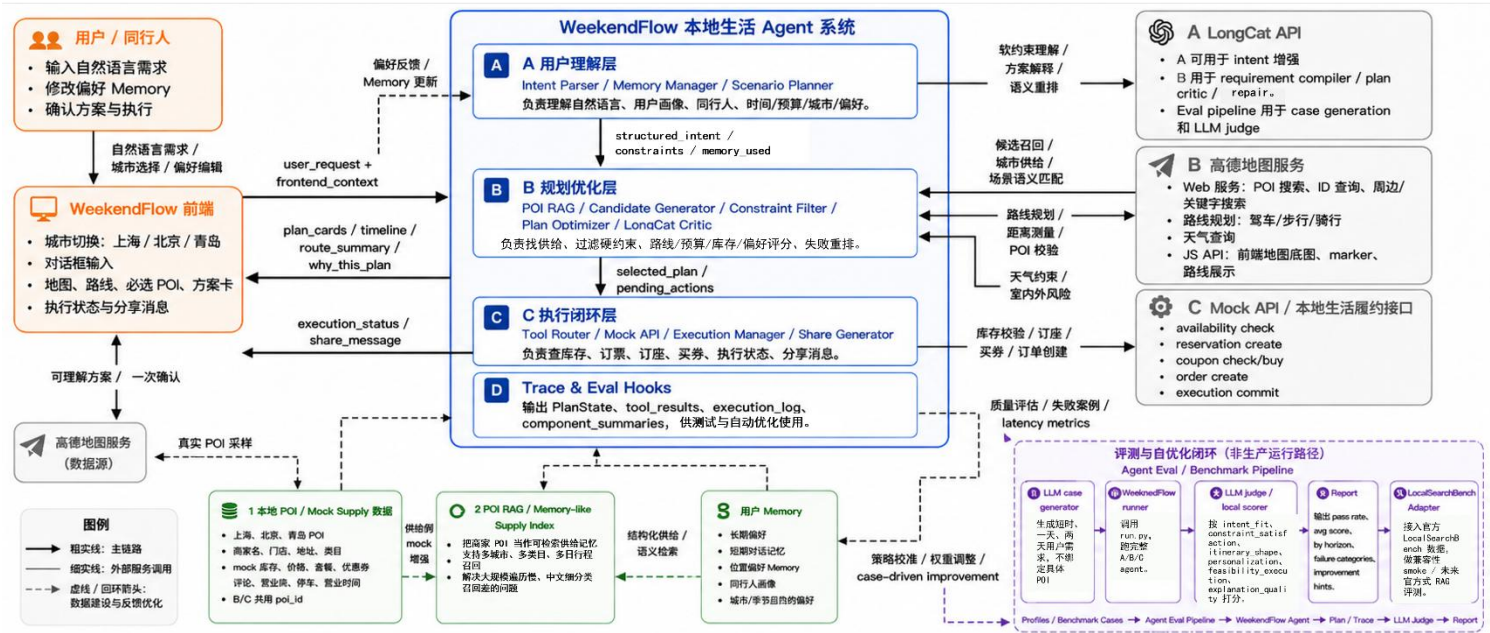
- Contract first：每阶段只依赖上游结构化字段，避免重复 NLP。
- Hard before soft：过敏、可达性、时间窗、人数、预算上限等硬约束优先于氛围、口味和新奇感。
- Evidence bound：推荐理由、执行动作和分享文案只能引用已选方案、候选证据和 mock API 结果。

## 关键创新点

- Value Probe 量化用户价值观**  
WeekendFlow 不只解析用户想做什么，还通过 Value Probe 抽取 family\_care、health、convenience、cost\_sensitivity 等隐含价值信号，并保留 evidence span，使“带孩子”“老婆减脂”“别折腾”“人均预算”可以进入 Memory、排序权重和解释链路。
- Current-first Memory Merge**  
系统采用 当前明确需求 > 当前 value 信号 > 近期 Memory > 长期 Memory > 默认画像 的优先级。Memory 只补充当前缺口，不能覆盖当前表达，避免历史偏好导致误个性化。
- RAG + 约束优化 + LLM Critic 混合格规**  
RAG 负责召回真实 POI，规则/约束优化负责时间、路线、预算、库存等硬约束，动态评分负责排序，LongCat 作为 critic 识别体验风险并提出有边界的重规划建议，避免 LLM 直接编造方案。

框架整体流程

WeekendFlow：从用户意图到本地生活执行闭环



异常处理机制

本系统采用以下策略：

- 超时：LLM、RAG、mock API、支付各自设置 deadline；超时视为依赖失败，不阻塞整条链路无限等待。
- 有限重试：只在单一层重试可幂等读请求；写入、预约、支付类动作必须依赖 idempotency key 或禁止自动重试。
- 退避与 jitter：短暂依赖失败可按 capped exponential backoff 重试，加入 jitter，避免同时重放请求。
- 降级：A-stage LLM 失败降级到规则解析；B-stage 证据不足降级到可解释的 partial plan 或 clarification；C-stage 执行失败保留规划结果和失败原因，不伪装成功。
- 熔断/跳过：当 execution\_ready=false、存在 blocking missing roles、或 B 明确要求 replan 时，tool\_router 不生成执行动作。
- 数据保护：错误日志必须脱敏 API key、支付信息和用户隐私；memory 写回仅记录规划相关事实与偏好。

观测与验收

每个节点向 tool\_results 和 execution\_log 写入摘要，关键字段包括 fallback reason、候选数量、过滤原因、selected plan score、execution status、retry history、payment status 和 final share message。

验收标准：

- 对缺失 slot、否定偏好、多人计数、跨城/本地混合、过敏与可达性约束有回归用例。
- LLM 关闭、LLM 超时、RAG 无结果、mock API 库存不足、支付未确认时，系统均能返回可解释状态。
- 不出现“上游未证实的信息被下游当成事实”的跨阶段污染。

测试结果

在 60 条上海、北京、青岛三地真实人类输入上统计

- 方案采纳率：88.6
- 预算满足率：93.1
- 用户偏好满足率：81.7
- 平均决策轮数：2.0
- 用户计划执行成功率：94.8